



40. Hidden Markov Models

"The present knows the past, but the secret stays hidden."

Previous Section:

 [39. Sequential Data Modeling](#)

Contents:

[1. Hidden Markov Models – The Overview](#)

[2. The Markov Chains](#)

[3. From Markov Chains to Hidden Markov Models](#)

[4. Hidden Markov Models – Two Probability Matrices](#)

[5. The Forward Algorithm](#)

[6. Hidden Markov Model with Python](#)

[Summary](#)

[References](#)

You've made it this far. Which means you're no longer just dealing with models that "look at data and make a prediction". You're stepping into something a bit more subtle. A bit more unsettling, if I'm being honest. Because most Machine Learning models ask a simple question: **"Given what I see... what should I predict?"** → But **Hidden Markov Models** don't quite think like that. They ask something different:

┆ **"What if what I see... isn't the real thing?"**

So here's where things start to shift. Up until now, we've been working with models that assume the data you see is the data that matters. Features are given. Labels are known. Even when things get complex, the model is still grounded in **observable reality**.

But Hidden Markov Models don't trust that. They operate under a very different assumption: The world you observe is only a shadow of something hidden underneath. And suddenly, the problem changes completely. You're no longer just predicting the next value. You're trying to **infer what you cannot see from what you can**.

Think about it like this. Imagine you're listening to footsteps behind a wall. You can't see the person. You don't know who they are. All you hear is the pattern: *step, step, pause, step*. From that alone, you start guessing: **Are they walking slowly? Are they tired? Are they hesitating?**

You're not observing the **state** directly. You're observing **signals generated by that state**. That is the core idea behind Hidden Markov Models.

And here's the part that makes it both powerful and difficult. Most models try to learn: **Input → Output**. Hidden Markov Models (HMMs) try to learn: **Hidden State → Observable Output** (and then reverse it...

to guess the hidden state)

So instead of saying: *“Given the present... predict the next”*

They are really saying:

“Given what I observe now... what is actually happening underneath... and what will it become next?”

This is why HMMs sit in a very strange place. They belong to Machine Learning—yes. But you’ll see them again in: Natural Language Processing; Speech Recognition; even parts of Deep Learning history. Because anywhere the **truth is hidden**, and only indirect signals exist, Hidden Markov Models start to make sense.

And I’ll be honest with you. This is where many people get lost. Not because the math is impossible, but because the mindset is different. You have to stop thinking in terms of: “What do I see?”. And start thinking in terms of: “What *could be causing* what I see?”

1. Hidden Markov Models – The Overview

Now let’s stop dancing around it and actually look at what this thing is. A **Hidden Markov Model** is not just another Machine Learning model you throw data into and hope it predicts something nice. It’s more like a way of thinking.

A way of saying: *“I know I’m not seeing the truth... but I still want to understand it.”*

At its core, a Hidden Markov Model is a **probabilistic sequential model** designed for sequences—data that unfolds over time. But unlike most models you’ve seen so far, it makes one very bold assumption: The thing that actually matters **is hidden**. What you observe? That’s just the surface.

Let me reframe it in a way I would actually think about it. You’re watching the world, collecting data, writing things down like a journalist. But you already know something is off. Because the data you collect is not the process itself—it’s just the **result of something deeper**.

So instead of asking: “What pattern do I see?” → You ask: “What kind of hidden system would produce this pattern?” → That is exactly what Hidden Markov Models are built for.

Observations vs Hidden States

Everything in HMM revolves around two parallel sequences:

- **Observations** → what you can actually see
- **Hidden States** → what is really happening underneath

Observations are easy. They’re concrete. Measurable. Recordable.

- Sound waves in speech
- Words in a sentence
- Movements in a gesture
- Exam scores of a student

These are the things you can point at and say: *“Yes, this happened.”*

Hidden states, on the other hand, are invisible. They are the **causes** behind what you observe.

- Sound → comes from a phoneme
- Words → come from grammatical roles

- Umbrellas → come from weather
- Scores → come from knowledge

You never see these directly. You **infer** them. And that leads to the real goal of the model:

“Given what I observe... what is the most likely sequence of hidden states behind it?”

Not prediction in the usual sense. Not classification in the usual sense. This is **reconstruction of an invisible process**.

The Markov Assumption (Where Things Get Convenient)

Now here's the trick that makes all of this even possible. Without simplification, modeling hidden processes over time would be a nightmare. So HMM makes a very specific assumption:

The present state depends only on the previous state.

Not the entire history. Not everything that ever happened. Just one step back.

$$P(state_t \mid state_1, state_2, \dots, state_{t-1}) = P(state_t \mid state_{t-1})$$

This is called the **Markov Property**. Is it perfectly realistic? No. Is it incredibly useful? Yes. Because it turns an impossible problem into something we can actually compute.

What the Model Actually Learns

Now let's strip away the abstraction. When you train a Hidden Markov Model, you're really estimating three things:

1. Transition Probabilities: $P(state_t \mid state_{t-1})$ — How the hidden world evolves. If I'm in one hidden state... how likely am I to move to another?

- Rainy → Sunny
- Understanding → Confusion
- Happy → Sad

This is the **flow of the hidden system over time**.

2. Emission Probabilities: $P(observation_t \mid state_t)$ — How the hidden world reveals itself

If I'm in a hidden state... what am I likely to observe?

- Rainy → Umbrellas
- High knowledge → High scores
- Sad → Shopping

This is the **bridge between invisible causes and visible effects**.

3. Initial Probabilities: $P(state_1)$ — Where everything begins

What is the system's starting point?

- Does it begin rainy?
- Does the student start strong?

Because even hidden systems have to start somewhere.

Why This Model Actually Matters

Most models need clean, complete information.

Hidden Markov Models don't.

They work when:

- The system is **partially observable**
- The data is **sequential**
- The truth is **hidden behind noise**

And that's a lot of real-world problems: Speech recognition; Natural Language Processing; Bioinformatics (DNA, proteins); Human behavior. In all of these cases, you don't see the real state. You see traces. And HMM says: "That's enough. I can work with traces."

The Three Problems That Define HMM

Everything you do with Hidden Markov Models boils down to three questions:

- **Evaluation** → *How likely is this observation sequence?*
- **Decoding** → *What hidden states most likely produced it?*
- **Learning** → *How do we learn the probabilities themselves?*

These aren't just steps. They are the entire framework.

A Simple Example (But Don't Underestimate It)

Let's say you're trying to understand someone. You cannot see their emotions directly.

But you observe their actions: **Walk; Shop; Clean. And** you suspect the hidden states are: **Happy; Sad; Angry**

Now imagine you observe this:

Walk → Shop → Clean → Walk → Walk → Shop → Clean → Walk → Clean → Shop

That's all you get. No labels. No explanations. Just behavior. And now comes the real challenge: What emotional state sequence could have generated this?

That's the HMM problem. Not: "Predict the next action" → But: "Explain the hidden story behind the actions"

And if this feels unfamiliar. Then good!!! Because this is the moment you stop treating data as truth, and start treating it as **evidence**.

2. The Markov Chains

Before we go any deeper, we need to step back. Because Hidden Markov Models didn't just appear out of nowhere. They are built on something much simpler. Something almost deceptively simple.

Markov Chains.

If Hidden Markov Models are about *hidden truth*... then Markov Chains are about something much more honest:

“Given where I am right now... where am I likely to go next?”

No hidden layers. No invisible states. Just transitions.

What is a Markov Chain?

A Markov Chain is a model of a system that **moves between states over time**. That's it. You define a set of possible states: Sad; Happy; Angry

And then you ask a single question:

“If I'm in one state now... what's the probability of moving to another state next?”

But here's the rule that changes everything. The model makes a very strong assumption:

The future depends only on the present, not the past. Not the full history. Not everything that happened before. Just the current state.

$$P(S_{t+1} \mid S_t, S_{t-1}, \dots, S_1) = P(S_{t+1} \mid S_t)$$

This is the **Markov Property**. And honestly, it feels like cheating. Because in real life, things *do* depend on history. But without this assumption? You wouldn't be able to model anything practical. So we accept it. And move on.

Thinking in Transitions

Now let's make this concrete. Suppose we're modeling someone's mood. The three states are Sad; Happy; Angry

Now instead of guessing randomly, we define probabilities of tomorrow's mood given today's mood.

What is the probability of being Sad tomorrow... given today's mood are...?

- $Happy \rightarrow P(\text{Sad} \mid \text{Happy})$
- $Sad \rightarrow P(\text{Sad} \mid \text{Sad})$
- $Angry \rightarrow P(\text{Sad} \mid \text{Angry})$

What is the probability of being Happy tomorrow... given today's mood are...?

- $Happy \rightarrow P(\text{Happy} \mid \text{Happy})$
- $Sad \rightarrow P(\text{Happy} \mid \text{Sad})$
- $Angry \rightarrow P(\text{Happy} \mid \text{Angry})$

What is the probability of being Angry tomorrow... given today's mood are...?

- $Happy \rightarrow P(\text{Angry} \mid \text{Happy})$
- $Sad \rightarrow P(\text{Angry} \mid \text{Sad})$
- $Angry \rightarrow P(\text{Angry} \mid \text{Angry})$

And once you define all of them, you've completely described how the system evolves over time.

The Transition Matrix

Now instead of listing probabilities everywhere, we organize them. Because yes, this *can* get messy. So we compress everything into a **transition matrix**.

- **Each row = current state; Each column = next state**

$$\begin{bmatrix} P(\text{Sad}|\text{Sad}) & P(\text{Happy}|\text{Sad}) & P(\text{Angry}|\text{Sad}) \\ P(\text{Sad}|\text{Happy}) & P(\text{Happy}|\text{Happy}) & P(\text{Angry}|\text{Happy}) \\ P(\text{Sad}|\text{Angry}) & P(\text{Happy}|\text{Angry}) & P(\text{Angry}|\text{Angry}) \end{bmatrix}$$

And there's one rule you cannot break: Each row must sum to 1. Because the system *must* go somewhere next.

A Real Example - Let's say we estimate the following:

$$\begin{bmatrix} 0.4 & 0.4 & 0.2 \\ 0.1 & 0.8 & 0.1 \\ 0.3 & 0.2 & 0.5 \end{bmatrix}$$

Now this isn't just numbers. This is behavior. This is a system revealing how it evolves.

Read it carefully:

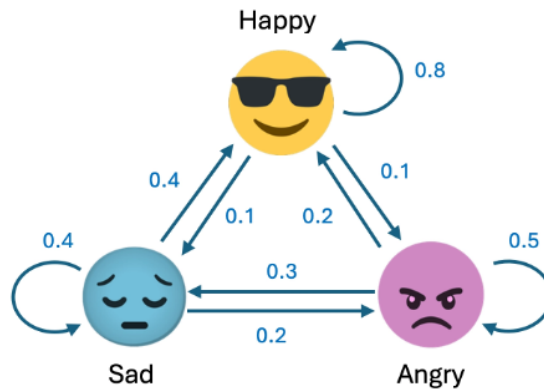
- If someone is **Happy**, there is
 - 80% chance they stay Happy
 - 10% chance they become Sad
 - 10% chance they become Angry
- If someone is **Sad**, there is
 - 40% chance they stay Sad
 - 40% chance they become Happy
 - 20% chance they become Angry
- If someone is **Angry**, there is
 - 50% chance they stay Angry
 - 30% chance they become Sad
 - 20% chance they become Happy

You're not predicting a single outcome. You're describing a **distribution of possible futures**.

The Graph Perspective

Now if you're like me, matrices are fine. But they don't *feel* intuitive. So we redraw the same idea as a graph.

- Each state → a node
- Each transition → a directed edge
- Each edge → labeled with probability



Now suddenly, you can see the system moving:

- $Sad \rightarrow Happy$ (0.4)
- $Happy \rightarrow Happy$ (0.8)
- $Angry \rightarrow Angry$ (0.5)

It becomes less like math, and more like watching behavior unfold.

What Markov Chains Really Are

Let me simplify everything we just did. A Markov Chain is a set of states; A rule for moving between them; And one assumption: The present is enough. No hidden variables. No indirect observations. Just: “Where am I now... and where can I go next?”

And this is exactly why we needed it. Because Hidden Markov Models will take this idea, and break it. They will say: “What if you don’t even know where you are right now?”

3. From Markov Chains to Hidden Markov Models

So far, we have worked with **Markov Chains**, where the key assumption is simple but powerful: we directly observe the states of the system.

For example, in a weather system, we might observe a clear sequence like: *Rainy* → *Rainy* → *Sunny* → *Sunny*

Everything is visible. The states are known at every step, and we can track how one state transitions into another using a transition matrix. But in many real-world problems... this assumption breaks. We don’t always observe the true state of the system. Instead, we only observe indirect signals—something generated by those hidden states. This is where **Hidden Markov Models (HMMs)** come in.

When States Become Hidden - Let’s revisit a simple example. We assume a person has internal **moods**: Happy; Sad; Angry.

However, we cannot directly observe these moods. Instead, we only observe **activities**: Walk; Shop; Clean. So the situation becomes:

Hidden State (Unseen)	Observation (Seen)
Happy	Walk
Sad	Shop
Angry	Clean

Here, the **mood is hidden**, but it still influences what we observe. This is the core idea of a Hidden Markov Model: ***Hidden states evolve like a Markov Chain, but they generate observable outputs.***

Core Idea of Hidden Markov Models - The Hidden Markov Model extends the Markov Chain in two layers:

1. State Transition Layer (Hidden Process)

- How hidden states evolve over time
- Follows the Markov property

2. Emission Layer (Observation Process)

- How hidden states generate observable outputs

So instead of only modeling:

$$P(\text{state}_t \mid \text{state}_{t-1})$$

We now also model:

$$P(\text{observation}_t \mid \text{state}_t)$$

Markov Assumption Still Holds. Even in HMMs, we still use the same Markov assumption.

Instead of depending on the full history:

$$P(\text{Day}_4 \mid \text{Day}_3, \text{Day}_2, \text{Day}_1)$$

We simplify it to:

$$P(\text{Day}_4 \mid \text{Day}_3)$$

This is what makes the system computationally feasible. For example:

$$P(\text{Angry} \mid \text{Sad}) = 0.2$$

Meaning: If the person is Sad today, there is a 20% chance they become Angry tomorrow.

Probability Distribution Over Time - Instead of tracking a single state, we track a **probability distribution**.

Let's assume we start with certainty: $P_1 = [0, 1, 0]$ → Meaning: $Sad = 0$; $Happy = 1$; $Angry = 0$

We use the same transition matrix:

$$A = \begin{bmatrix} 0.4 & 0.4 & 0.2 \\ 0.1 & 0.8 & 0.1 \\ 0.3 & 0.2 & 0.5 \end{bmatrix}$$

Computing Next State - Day 2:

$$P_2 = P_1 \cdot A = [0, 1, 0] \cdot A = [0.1, 0.8, 0.1]$$

So: $Sad = 0.1$; $Happy = 0.8$; $Angry = 0.1$. Even with certainty at the beginning, uncertainty naturally appears over time.

Similarly, for Day 3:

$$P_3 = P_2 \cdot A$$

Now we compute step by step:

- *Sad* : $0.1 \times 0.4 + 0.8 \times 0.1 + 0.1 \times 0.3 = 0.15$
- *Happy* : $0.1 \times 0.4 + 0.8 \times 0.8 + 0.1 \times 0.2 = 0.70$
- *Angry* : $0.1 \times 0.2 + 0.8 \times 0.1 + 0.1 \times 0.5 = 0.15$

So: $P_3 = [0.15, 0.70, 0.15]$

This means: The system is most likely *Happy*, but still uncertain between *Sad* and *Angry*.

We can simulate this process easily with Python:

```
import numpy as np

# Transition matrix
A = np.array([[0.4, 0.4, 0.2],
              [0.1, 0.8, 0.1],
              [0.3, 0.2, 0.5]])

# Initial state: Happy
p = np.array([0.0, 1.0, 0.0])

# Evolve for 10 steps
for i in range(10):
    p = np.dot(p, A)
    print(p)
```

```
[0.1 0.8 0.1]
[0.15 0.7 0.15]
[0.175 0.65 0.175]
[0.1875 0.625 0.1875]
[0.19375 0.6125 0.19375]
[0.196875 0.60625 0.196875]
[0.1984375 0.603125 0.1984375]
[0.19921875 0.6015625 0.19921875]
[0.19960938 0.60078125 0.19960938]
[0.19980469 0.60039063 0.19980469]
```

This shows how probability distributions evolve over time until they stabilize.

Long-Term Behavior (Stationary Distribution)

If we keep multiplying repeatedly, the system eventually converges to a stable distribution:

$$\pi \approx [0.2, 0.6, 0.2]$$

This is called the **stationary distribution**. It means: Sad $\approx 20\%$; Happy $\approx 60\%$; Angry $\approx 20\%$

A key property: The stationary distribution does not depend on the initial state. No matter where we start, the system eventually stabilizes.

Now we extend the idea. In a Hidden Markov Model: We **do not observe the states directly**; We only observe outputs generated from those states

So instead of seeing: *Happy* → *Sad* → *Angry*. We see: *Walk* → *Shop* → *Clean*

But behind this sequence, a hidden process is evolving.

Emission Probabilities - Each hidden state generates observations with certain probabilities. For example:

Hidden State	Observation Probability
Happy	<i>Walk</i> (0.7), <i>Shop</i> (0.2), <i>Clean</i> (0.1)
Sad	<i>Walk</i> (0.2), <i>Shop</i> (0.6), <i>Clean</i> (0.2)
Angry	<i>Walk</i> (0.1), <i>Shop</i> (0.3), <i>Clean</i> (0.6)

So we now have two systems:

- **Transition model** → how moods change
- **Emission model** → how moods produce behavior

Once we define these probabilities, we can solve key problems:

- **Forward-Backward Algorithm**: Computes probability of hidden states given observations
- **Viterbi Algorithm**: Finds the most likely hidden state sequence
- **Baum-Welch Algorithm**: Learns transition + emission probabilities from data

These algorithms all build on the same idea: **hidden structure + observed data** → **probabilistic inference**

We can now clearly summarize:

- Markov Chain → models visible state transitions
- Hidden Markov Model → models hidden state transitions + visible outputs

So HMM is not a replacement—it is an extension. It allows us to model situations where the true state of the system is invisible, but its effects are observable.

Now that we understand how hidden states evolve and generate observations, the next step is to learn:

┃ *How do we compute the most likely hidden sequence from observations?*

This leads us directly to **HMM key probability matrices and inference algorithms**.

4. Hidden Markov Models - Two Probability Matrices

Now that we understand what a Hidden Markov Model *is*, let's open it up and look inside. Because at its core, an HMM is not magic. It's just **two probability matrices working together**.

That's it. But those two matrices? They define everything.

The Two Layers of an HMM

A Hidden Markov Model has two fundamental components:

- **Transition Matrix** → how hidden states evolve
- **Emission Matrix** → how hidden states produce observations

Think of it like this:

- Hidden world (moods) → evolves over time
- Visible world (activities) → generated from that hidden world

Transition Matrix (State → State)

Let's start with the first one. The **transition matrix** describes how hidden states change over time. Formally:

$$A_{ij} = P(S_{t+1} = j \mid S_t = i)$$

This means: If I am in state i today... what is the probability I move to state j tomorrow?

Example: Mood Transitions - Let's reuse our hidden states: *Sad*; *Happy*; *Angry*

Then the transition matrix is:

$$A = \begin{bmatrix} 0.4 & 0.4 & 0.2 \\ 0.1 & 0.8 & 0.1 \\ 0.3 & 0.2 & 0.5 \end{bmatrix}$$

Each row represents the current state; Each column represents the next state. For example:

$$P(\text{Happy} \mid \text{Sad}) = 0.4$$

$$P(\text{Angry} \mid \text{Sad}) = 0.2$$

$$P(\text{Happy} \mid \text{Happy}) = 0.8$$



Important Constraint → Each row must sum to 1: $\sum_j A_{ij} = 1$. Because: From any state, you must go somewhere.

Emission Matrix (State → Observation)

Now comes the second matrix. This one is what makes HMMs different from Markov Chains. The **emission matrix** tells us:

$$B_{ij} = P(O_t = j \mid S_t = i)$$

Meaning: If the system is in hidden state i , what is the probability it produces observation j ?

Example: Mood → Activity. Observations: *Walk*; *Shop*; *Clean*.

Then the emission matrix is:

$$B = \begin{bmatrix} 0.1 & 0.2 & 0.7 \\ 0.6 & 0.3 & 0.1 \\ 0.2 & 0.6 & 0.2 \end{bmatrix}$$

Rows = hidden states (*Sad*, *Happy*, *Angry*); Columns = observations (*Walk*, *Clean*, *Shop*)

Let's read it like a story:

$$P(\text{Walk} \mid \text{Happy}) = 0.6$$

→ If someone is *Happy*, they are likely to walk.

$$P(\text{Shop} \mid \text{Sad}) = 0.7$$

→ If someone is *Sad*, they go shopping (classic).

$$P(\text{Clean} \mid \text{Angry}) = 0.6$$

→ If someone is *Angry*, they would likely do some cleaning



Constraint Again - Each row must sum to 1: $\sum_j B_{ij} = 1$. Because: A state must produce some observation.

Two Layers — One Model

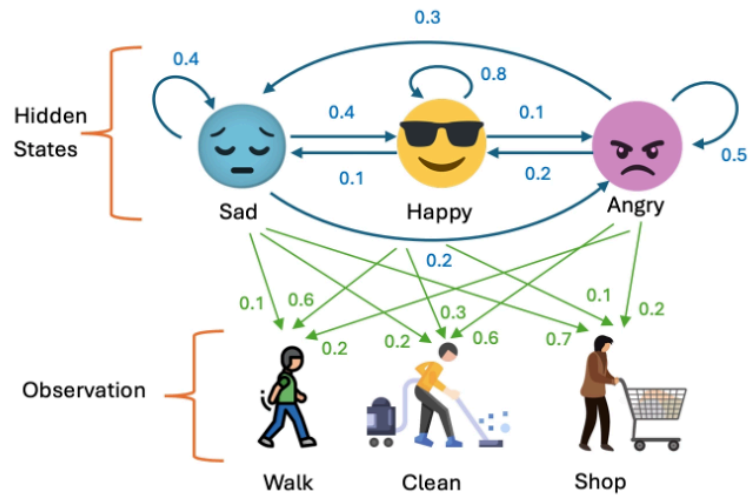
Now combine both matrices. You get a **two-layer system**:

1. Hidden layer → evolves using A
2. Observation layer → generated using B

Graphically:

- Nodes = hidden states
- Edges between states = transition probabilities
- Edges to observations = emission probabilities

So the system looks like: Hidden chain → generates visible sequence



Transition and emission probabilities can be visualized through a graph where nodes represent hidden states like *Sad*, *Happy*, or *Angry*.

In this model, **edges between states show transition probabilities**, while **edges connecting states to observations illustrate emission probabilities**.

Full Definition of an HMM

A Hidden Markov Model is fully defined by **three components**:

1. Initial state distribution

$$\pi = P(X_1)$$

2. Transition matrix

$$A_{ij} = P(X_{t+1} = j \mid X_t = i)$$

3. Emission matrix

$$B_{ij} = P(Y_t = j \mid X_t = i)$$

Matrix Representation (Clean Form) → Let's write everything neatly:

$$\text{Transition Matrix } A = \begin{bmatrix} 0.4 & 0.4 & 0.2 \\ 0.1 & 0.8 & 0.1 \\ 0.3 & 0.2 & 0.5 \end{bmatrix}$$

$$\text{Emission Matrix } B = \begin{bmatrix} 0.1 & 0.2 & 0.7 \\ 0.6 & 0.3 & 0.1 \\ 0.2 & 0.6 & 0.2 \end{bmatrix}$$

Full Joint Probability Structure

Now, this is the part where everything comes together. Let's define:

- $X_t \rightarrow$ hidden state at time t
- $Y_t \rightarrow$ observation at time t

Step-by-Step Construction

For Day 1:

$$P(X_1) \cdot P(Y_1 | X_1)$$

For Day 2:

$$P(X_2 | X_1) \cdot P(Y_2 | X_2)$$

For Day 3:

$$P(X_3 | X_2) \cdot P(Y_3 | X_3)$$

Full Sequence (General Form)

$$P(X_1) \cdot P(Y_1 | X_1) \cdot P(X_2 | X_1) \cdot P(Y_2 | X_2) \cdots P(X_T | X_{T-1}) \cdot P(Y_T | X_T)$$

Example (10 Days)

$$P(X_1) \cdot P(Y_1|X_1) \cdot P(X_2|X_1) \cdot P(Y_2|X_2) \cdots P(X_{10}|X_9) \cdot P(Y_{10}|X_{10})$$

This formula combines: Initial probability + Transition probabilities + Emission probabilities Into one single expression. And this... is the **core engine of HMMs**.

If Markov Chains were about: "Where do we go next?" → Then Hidden Markov Models are about: "What hidden process explains what we see?". You don't observe the truth. You observe *evidence*. And your job? Reconstruct the hidden story behind it.

Now that we understand how HMMs are built...

The real question becomes: Given only observations... how do we actually compute probabilities efficiently? Because if you try to compute that big joint probability directly... you'll quickly realize: It explodes exponentially.

And that's exactly why we need: **The Forward Algorithm**.

5. The Forward Algorithm

We reach the moment where things get *real*. Up until now, everything looked clean:

- We defined transition probabilities
- We defined emission probabilities
- We even wrote the full joint probability

But there's a problem. A serious one.

In real-world applications, we **do not know the hidden states**. We only observe:

$$O = (O_1, O_2, O_3, \dots, O_T)$$

For example:

$$O = [\text{Walk, Shop, Clean, Walk, Walk, Shop, Clean, Walk, Clean, Shop}]$$

But the hidden sequence: $X_1, X_2, X_3, \dots, X_T$ is completely unknown.

If we try to compute:

$$P(O_1, O_2, \dots, O_T)$$

We must sum over **all possible hidden state sequences**:

$$P(O) = \sum_X P(X, O)$$

And here's the problem: The number of possible hidden sequences grows exponentially. If we have: N states, T time steps. Then the number of sequences is: N^T → Even for small values, this explodes. So brute force? Not happening.

The Solution: Dynamic Programming

Instead of recomputing everything, we reuse computations. This idea leads to the **Forward Algorithm**.

What the Forward Algorithm Computes → The goal is simple: $P(O_1, O_2, \dots, O_T)$. The probability of observing the entire sequence, regardless of which hidden states produced it.

We define a new quantity:

$$\alpha_t(i) = P(O_1, O_2, \dots, O_t, X_t = i)$$

Meaning: The probability of observing the sequence up to time t , and ending in state i . This is the heart of the algorithm.

• Step 1 — Initialization (Day 1)

We begin with:

$$\alpha_1(i) = \pi_i \cdot B_i(O_1)$$

Where: $\pi_i = P(X_1 = i)$; $B_i(O_1) = P(O_1 \mid X_1 = i)$

Given a uniform initial distribution:

$$\pi = \left[\frac{1}{3}, \frac{1}{3}, \frac{1}{3} \right]$$

First observation: $O_1 = \text{Walk}$. From emission matrix:

$$B(\text{Sad}, \text{Walk}) = 0.1; \quad B(\text{Happy}, \text{Walk}) = 0.6; \quad B(\text{Angry}, \text{Walk}) = 0.2$$

Compute:

$$\alpha_1(\text{Sad}) = \frac{1}{3} \times 0.1 = 0.0333; \quad \alpha_1(\text{Happy}) = \frac{1}{3} \times 0.6 = 0.2; \quad \alpha_1(\text{Angry}) = \frac{1}{3} \times 0.2 = 0.0667$$

Result: $\alpha_1 = [0.0333, 0.2, 0.0667]$

• Step 2 — Recursion (Day 2)

Now we move forward. For $t \geq 2$:

$$\alpha_t(j) = \left(\sum_i \alpha_{t-1}(i) \cdot A_{ij} \right) \cdot B_j(O_t)$$

Given: $\alpha_1 = [0.0333, 0.2, 0.0667] \rightarrow$ Second observation: $O_2 = \text{Shop}$

Compute $\alpha_2(\text{Sad})$

$$\text{Transition sum : } 0.0333 \times 0.4 + 0.2 \times 0.1 + 0.0667 \times 0.3 = 0.05333$$

$$\text{Emission : } B_{\text{Sad}}(\text{Shop}) = 0.7$$

$$\text{Final : } \alpha_2(\text{Sad}) = 0.7 \times 0.05333 = 0.037331$$

Compute $\alpha_2(\text{Happy})$

$$\text{Transition sum : } 0.2 \times 0.8 + 0.0333 \times 0.4 + 0.0667 \times 0.2 = 0.18666$$

$$\text{Emission : } B_{\text{Happy}}(\text{Shop}) = 0.1$$

$$\text{Final : } \alpha_2(\text{Happy}) = 0.01867$$

Compute $\alpha_2(\text{Angry})$

$$\text{Transition sum : } 0.0667 \times 0.5 + 0.2 \times 0.1 + 0.0333 \times 0.2 = 0.06001$$

$$\text{Emission : } B_{\text{Angry}}(\text{Shop}) = 0.2$$

$$\text{Final : } \alpha_2(\text{Angry}) = 0.012002$$

Result: $\alpha_2 = [0.037331, 0.01867, 0.012002]$

Step 3 — Recursion (Day 3)

Same formula:

$$\alpha_t(j) = \left(\sum_i \alpha_{t-1}(i) A_{ij} \right) B_j(O_t)$$

Given: $\alpha_2 = [0.037331, 0.01867, 0.012002] \rightarrow O_3 = \text{Clean}$

Compute $\alpha_3(\text{Sad})$

$$\text{Transition sum : } 0.037331 \times 0.4 + 0.01867 \times 0.1 + 0.012002 \times 0.3 = 0.0204$$

$$\text{Emission : } B_{\text{Sad}}(\text{Clean}) = 0.2$$

$$\text{Final : } \alpha_3(\text{Sad}) = 0.00408$$

Compute $\alpha_3(\text{Happy})$

$$\text{Transition sum : } 0.01867 \times 0.8 + 0.037331 \times 0.4 + 0.012002 \times 0.2 = 0.0322688$$

$$\text{Emission : } B_{\text{Happy}}(\text{Clean}) = 0.3$$

$$\text{Final : } \alpha_3(\text{Happy}) = 0.00968064$$

Compute $\alpha_3(\text{Angry})$

Transition sum : $0.012002 \times 0.5 + 0.01867 \times 0.1 + 0.037331 \times 0.2 = 0.0153342$

Emission : $B_{\text{Angry}}(\text{Clean}) = 0.6$

Final : $\alpha_3(\text{Angry}) = 0.00920052$

Result: $\alpha_3 = [0.00408, 0.00968064, 0.00920052]$

Continuing the Process - From Day 4 to Day 10, nothing changes. We repeat:

$$\alpha_t(j) = \left(\sum_i \alpha_{t-1}(i) A_{ij} \right) B_j(O_t)$$

Each step: Uses previous $\alpha \rightarrow$ Applies transitions $\rightarrow$ Applies emissions. This is **efficient propagation** of probabilities.

Final Step — Total Probability

After computing α_T , we sum:

$$P(O_1, O_2, \dots, O_T) = \sum_{j=1}^N \alpha_T(j)$$

Why Summation? Because the sequence can end in *any* hidden state. So we must consider all possibilities.

If you step back, the Forward Algorithm is doing something very elegant: Instead of guessing every hidden path... it *compresses all possibilities into a single evolving vector*. At each step: It carries forward all possible histories. Weighs them by probability. And merges them efficiently

┆ *"We don't try to guess the hidden story... We let probability quietly accumulate it for us."*

We can now compute: $P(\text{observations})$. But a new question appears: What if we want the *most likely hidden sequence*? Not the probability... but the actual path. And that leads us to **The Viterbi Algorithm**.



The **Viterbi Algorithm** answers a different question from the Forward Algorithm. Instead of computing the total probability of an observation sequence, it tries to find the **single most likely sequence of hidden states** that could have generated those observations. In other words, given what we observe, it asks: *"What is the most probable hidden story behind it?"* Formally, it computes:

$$\arg \max_{X_1, \dots, X_T} P(X_1, X_2, \dots, X_T \mid O_1, O_2, \dots, O_T)$$

Rather than summing over all possible paths (like the Forward Algorithm), the Viterbi Algorithm uses dynamic programming to keep track of the **best path leading to each state at every time step**, along with back-pointers to reconstruct the optimal sequence at the end.

Conceptually, you can think of it as a “**maximum probability path finder**” through the hidden state space. At each time step, it selects the most likely transition into each state and discards less probable paths, making the computation efficient even for long sequences. This makes the Viterbi Algorithm especially useful in tasks where we care about the actual hidden structure, such as **part-of-speech tagging in NLP, speech recognition, and sequence labeling problems**. While we won’t dive into its full derivation here, it builds directly on the same transition and emission framework of Hidden Markov Models—and you’ll often see it reappear later in more advanced topics like NLP and Deep Learning.

Forward Algorithm using Python

```
import numpy as np

def forward_algorithm(obs, hidden_states, transition_matrix, emission_matrix,
                     init=np.array([1/3, 1/3, 1/3])):

    T = len(obs)      # number of time steps
    N = len(hidden_states) # number of states

    alpha = np.zeros((T, N))

    # Initialization (t = 1)
    O0 = obs_map[obs[0]]
    alpha[0, :] = init * emission_matrix[:, O0]
    print("alpha[0]:", alpha[0])

    # Recursion
    for t in range(1, T):
        Ot = obs_map[obs[t]]
        for j in range(N):
            alpha[t, j] = (np.sum(alpha[t-1, :] * transition_matrix[:, j])
                           * emission_matrix[j, Ot])
        print(f"alpha[{t}]:", alpha[t])

    # Final probability
    P_0 = np.sum(alpha[T-1, :])
    print("\nProbability of the observation sequence:", P_0)
```

```

observations = ["Walk", "Shop", "Clean", "Walk", "Walk",
               "Shop", "Clean", "Walk", "Clean", "Shop"]

hidden_states = ["Sad", "Happy", "Angry"]

obs_map = {"Walk": 0, "Clean": 1, "Shop": 2}

A = np.array([[0.4, 0.4, 0.2],
              [0.1, 0.8, 0.1],
              [0.3, 0.2, 0.5]])

B = np.array([[0.1, 0.2, 0.7],
              [0.6, 0.3, 0.1],
              [0.2, 0.6, 0.2]])

pi = np.array([1/3, 1/3, 1/3])

forward_algorithm(observations, hidden_states, A, B, pi)

```

```

alpha[0]: [0.03333333 0.2      0.06666667]
alpha[1]: [0.03733333 0.01866667 0.012    ]
alpha[2]: [0.00408  0.00968 0.0092   ]
alpha[3]: [0.000536 0.0067296 0.0012768]
alpha[4]: [0.00012704 0.00351206 0.00028371]
alpha[5]: [0.000341 0.00029172 0.00010369]
alpha[6]: [3.93356800e-05 1.17154099e-04 8.95309056e-05]
alpha[7]: [5.43089536e-06 7.64182395e-05 1.28695997e-05]
alpha[8]: [2.73501240e-06 1.97642609e-05 9.09768174e-06]
alpha[9]: [4.05981490e-06 1.87249500e-06 1.41445389e-06]

```

Probability of the observation sequence: 7.3467637917696e-06

At each time step, the algorithm prints:

$$\alpha_t = [\alpha_t(\text{Sad}), \alpha_t(\text{Happy}), \alpha_t(\text{Angry})]$$

Where:

$$\alpha_t(i) = P(O_1, O_2, \dots, O_t, X_t = i)$$

Day 1 Example

$$\alpha_1 = [0.0333, 0.2, 0.0667]$$

Interpretation: Highest value = *Happy* → Why? Because: $P(\text{Walk} \mid \text{Happy}) = 0.6$. So after seeing *Walk*, the model leans toward *Happy*.

As Time Progresses → Values become smaller: $\alpha_5 \approx [0.000127, 0.003512, 0.000284]$

This is normal. We are multiplying probabilities over longer sequences. But the **relative values still matter**:

- “*Happy*” remains dominant
- Because multiple “*Walk*” observations appear

Final Step (Day 10)

$$\alpha_{10} = [4.06 \times 10^{-6}, 1.87 \times 10^{-6}, 1.41 \times 10^{-6}]$$

Meaning:

$$P(O, X_{10} = \text{Sad}) > P(O, X_{10} = \text{Happy}) > P(O, X_{10} = \text{Angry})$$

So if the sequence ends somewhere, ***Sad* is the most likely final state**

Final Probability - We sum over all final states:

$$P(O_1, O_2, \dots, O_T) = \sum_{j=1}^N \alpha_T(j)$$

Result:

$$P(O) \approx 7.35 \times 10^{-6}$$

The Forward Algorithm doesn’t guess a single path. It quietly accumulates *all possible paths*, and compresses them into one number.

What Problem Did We Solve? This answers the **Evaluation Problem**: “*Given a model and observations... how likely is this sequence?*” → And we solved it without exponential explosion.

6. Hidden Markov Model with Python

Theory is nice. But at some point, we stop deriving equations, and let the model **learn by itself**. This is where things get interesting.

(Run this code to see the Initial probabilities; Transition Matrix; and Emission Matrix learned by the HMM model)

```
import numpy as np
from hmmlearn import hmm # Install this package in case you are required to

observations = ["Walk", "Shop", "Clean", "Walk", "Walk",
               "Shop", "Clean", "Walk", "Clean", "Shop"]

# Map observations to numbers
obs_map = {"Walk": 0, "Clean": 1, "Shop": 2}
obs_seq = np.array([obs_map[o] for o in observations]).reshape(-1, 1)

print("\nHidden Markov Model:")
model = hmm.CategoricalHMM(n_components=3, n_iter=100, tol=0.01, verbose=True)

# Learn everything from data
model.fit(obs_seq)
```

```
# Learned parameters
print("Learned start probabilities (pi):\n", model.startprob_)
print("\nLearned transition matrix (A):\n", model.transmat_)
print("\nLearned emission matrix (B):\n", model.emissionprob_)

# Decode hidden states
log_prob, hidden_states = model.decode(obs_seq, algorithm="viterbi")
print("\nMost likely hidden states sequence:", hidden_states)
```

Let's break it down simply. We observe a sequence of activities. We **don't know the hidden states**. We don't even know π, A, B

So what do we do? We let the model figure everything out.

Key Steps

- Convert observations $\rightarrow$ numbers (because models don't understand words)
- Use **Categorical HMM** $\rightarrow$ for discrete data
- Set:
 - `n_components=3` $\rightarrow$ number of hidden states
 - `n_iter` $\rightarrow$ training iterations
 - `tol` $\rightarrow$ convergence threshold

How the Model Learns

The training uses:

Expectation-Maximization (EM) $\Rightarrow$ Baum-Welch Algorithm

It iteratively estimates: Initial probabilities $\pi \rightarrow$ Transition matrix $A \rightarrow$ Emission matrix B . Until convergence.

After Training, we get:

- $\pi \rightarrow$ where the system starts
- $A \rightarrow$ how hidden states evolve
- $B \rightarrow$ how observations are generated

And finally, we decode the hidden sequence using the **Viterbi Algorithm** $\rightarrow$ most likely hidden path

Important Reality Check - There's something you *must* understand here. This example uses only **10 observations**. But the model has: Start probabilities; Transition matrix; Emission matrix $\rightarrow$ **Many parameters to estimate**

So you may see warnings like: **"Model has more parameters than data..."** $\rightarrow$ This leads to:

Overfitting; Extreme probabilities (close to 0 or 1); Unstable results. This is called a **degenerate solution**.

The model fits the data, but might not *understand* it. In real applications: HMMs need **large datasets** to learn meaningful patterns.

This entire section shows something subtle but powerful: Even when we don't know the hidden states, the model can reconstruct them from observations alone. That's the essence of **unsupervised learning**.

Summary

Let's compress everything into the core ideas. **Hidden Markov Models** is a probabilistic model for **sequential data**. Built on **hidden states + observable outputs**. Defined by three components: π, A, B

The Forward Algorithm

- Computes: $P(O_1, O_2, \dots, O_T)$
- Solves the **evaluation problem**
- Uses dynamic programming to avoid exponential complexity

The Big Picture

- Markov Chains → visible states
- HMMs → hidden states + observations
- Forward Algorithm → probability of sequence
- Viterbi Algorithm → most likely hidden path
- Baum-Welch → learn parameters from data

"You never see the system directly... you only see what it produces. And from that... you reconstruct the hidden truth."

(The Viterbi Algorithm deserves its own spotlight—and we'll meet it again where it truly shines, like POS tagging in NLP.)

References

<https://www.geeksforgeeks.org/machine-learning/hidden-markov-model-in-machine-learning/>

<https://www.mathworks.com/help/stats/hidden-markov-models-hmm.html>

<https://inst.eecs.berkeley.edu/~cs188/textbook/hmms/hmm.html>

<https://builtin.com/articles/hidden-markov-model>

<https://www.youtube.com/watch?v=RWkHJnFj5rY>

<https://www.youtube.com/watch?v=i3AkTO9HLXo>

<https://www.youtube.com/watch?v=9-sPm4CfcD0>

Next Section:

 [41. Gaussian Mixture Models](#)